Лабораторная работа №2. Параллельное вычисление определенного интеграла методом средних прямоугольников

1. Описание работы

На языке C реализованы последовательная и параллельная версии программы вычисления интеграла функции $f(x) = (x^2 - 1)\cdot 10^{-2x}$ на отрезке [0; 1] методом средних прямоугольников. Параллельная версия использует директиву #pragma omp parallel for reduction(+:sum), время выполнения измеряется функцией omp_get_wtime(), а для вычисления с заданной точностью n удваивается до выполнения условия $|I_{2n} - I_n| < \varepsilon$. Эксперименты проводились на процессоре Intel Core i5-12450H (4 P-ядра, 4 E-ядра, 12 логических потоков) в WSL Ubuntu с компилятором GCC.

2. Результаты экспериментов

Таблица 1 – Эксперимент 1. Влияние количества потоков (при $n = 10^8$)

Потоки	Время, с	Ускорение
1	0.995935	1.01
2	0.640438	1.57
4	0.476278	2.11
8	0.373256	2.69
16	0.334021	3.01

Таблица 2 – Эксперимент 2. Влияние числа разбиений (при 8 потоках)

n	Значение интеграла	Время, с
10^4	-0.197816825265645	0.001181
10^5	-0.197816827157027	0.000484
10^6	-0.197816827175943	0.004768
10^7	-0.197816827176130	0.052723
10^8	-0.197816827176145	0.451891

3. Результат вычисления с заданной точностью

Минимальное число разбиений, при котором достигается заданная точность (10^{-10}), равно 81920. Время выполнения последовательной версии – 0.002862 с., параллельной (8 потоков) – 0.001306 с.

4. Вывод

С увеличением числа потоков время выполнения уменьшается: при $n = 10^8$ один поток считал интеграл 0,99 с, а 16 потоков – 0,33 с. Но каждый новый поток даёт меньше эффективности: 8 потоков ускорили программу только в 2,69 раза, так как в моём процессоре только 4 производительных ядра. Чем больше разбиений, тем точнее результат: при увеличении n в 10 раз ошибка уменьшалась примерно в 100 раз (с $2\cdot 10^{-9}$ при $n = 10^4$ до $2\cdot 10^{-11}$ при $n = 10^5$). Reduction здесь эффективен, так как значения $f(x_i)$ вычисляются независимо и общей является только сумма. Каждый поток считает свою локальную сумму без блокировок, а частичные суммы складываются один раз в конце, тогда как critical или atomic требовали бы синхронизации на каждой итерации.